

REUNIÃO INTERMÉDIA 2 · MAIO 2026

AI Based Compilation for Custom Hardware — Atualização

Deniz Günes

Projeto Integrador 25/26, L.EIC, FEUP

Abordagem anterior: usar a GNN do LISA para gerar labels (*spatial/temporal distance*) e fazer **pruning** de literais no SAT-MapIt antes do solve.

- Conversão de DFGs sintéticos para o formato do LISA
- GNN treinada para produzir 4 labels por nó
- Adicionavam-se cláusulas $\neg(\text{lit}_s \wedge \text{lit}_d)$ quando a distância real excedia a previsão da GNN
- Resultado preliminar: 48% de literais pruned, mas redução era essencialmente geométrica

SAT-MapIt

- Construído em torno de *modulo scheduling*
- Assume um *Initiation Interval* (II) e coloca operações ao longo de ciclos
- Reasoning sobre branches, *live-ins/live-outs*, KMS

LISA

- Não emite diretamente um KMS ou schedule de *modulo scheduling*
- Prediz labels como ordem de scheduling e distâncias espaciais/temporais
- Vocabulário de operações mais reduzido

Conclusão

Por simplicidade e devido ao tempo limitado para o projeto integrador, simplificamos o problema de mapeamento para um cenário de *one-shot mapping* sem II, KMS ou *modulo scheduling*.

Em vez de forçar LISA dentro do SAT-MapIt, **invertemos** a ideia:

1. Pegar num DFG grande
2. Correr a GNN para obter features por nó
3. Usar essas features para **partir** o grafo em subgrafos
4. Materializar dependências cortadas com operações de memória sintéticas
5. Fazer **one-shot mapping** de cada partição com um SAT mapper simplificado

Resultado: decomposição temporal entre partições, *mapping* espacial dentro de cada uma. Sem II, sem KMS, sem *modulo scheduling*.

5 stages

DFG generation → **LISA-GNN inference** → **Partitioning** → **Materialisation**
→ **SAT mapping**

- **DFG generation** — geradores sintéticos (`dfg_gen`) a partir de templates de operações (e.g. matrix multiplication, convolution)
- **LISA-GNN** — prediz `label0` (prioridade por nó) usada pelo particionador *greedy*
- **Partitioning** — *greedy* guiado pela GNN
- **Materialisation** — cortes de produtores não-load viram `u -> synthetic_store` e `synthetic_load -> v`; cortes vindos de load são recarregados na partição destino
- **SAT mapping** — Z3 verifica validade espacial de cada partição

Quando o DFG não cabe num único *config* do CGRA, dividimos em $P_1, \dots, P_K$, impondo:

$$\text{partition}(\text{src}) \leq \text{partition}(\text{dst})$$

para cada aresta $\text{src} \rightarrow \text{dst}$, evitando dependências que voltariam atrás.

Greedy partitioner:

- Itera sobre nós *ready*
- Prioridades hand-crafted: **ASAP**, **ALAP**, *edge count*
- Critério de qualidade: reduzir arestas cortadas (menos tráfego sintético)
- Pode usar as predições `label0` da GNN como prioridade

Para cada partição, $p_{n,e} = 1$ sse o nó n está no PE e . Cláusulas:

- (C1) compatibilidade de operação: $\text{op}(n) \notin \text{ops}(e) \Rightarrow \neg p_{n,e}$
- (C2) exatamente um PE por nó: $\sum_e p_{n,e} = 1$
- (C3) no máximo uma instrução por PE: $\sum_n p_{n,e} \leq 1$
- (C4) dependências em ciclo vizinho:

$$\forall (u, v) \in E, \quad \bigvee_{a,b: \text{nbr}(a,b)} (p_{u,a} \wedge p_{v,b})$$

- (C5) PEs com acesso a memória para load/store

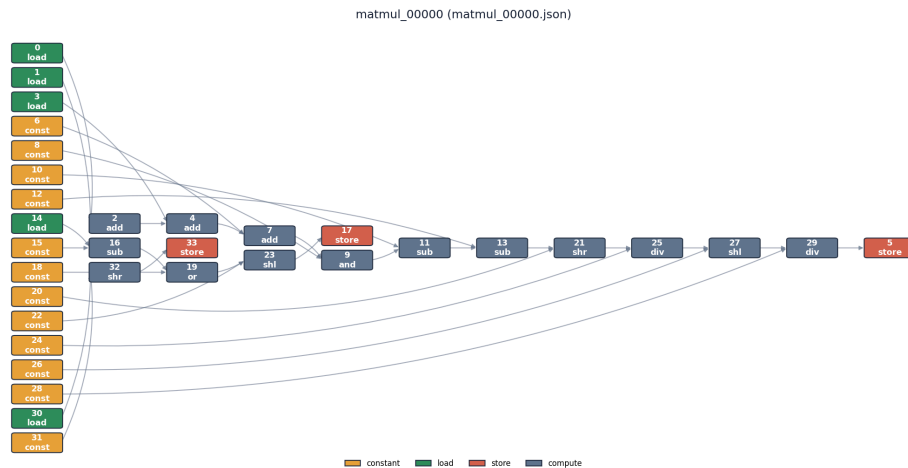


Figure 1: DFG sintético

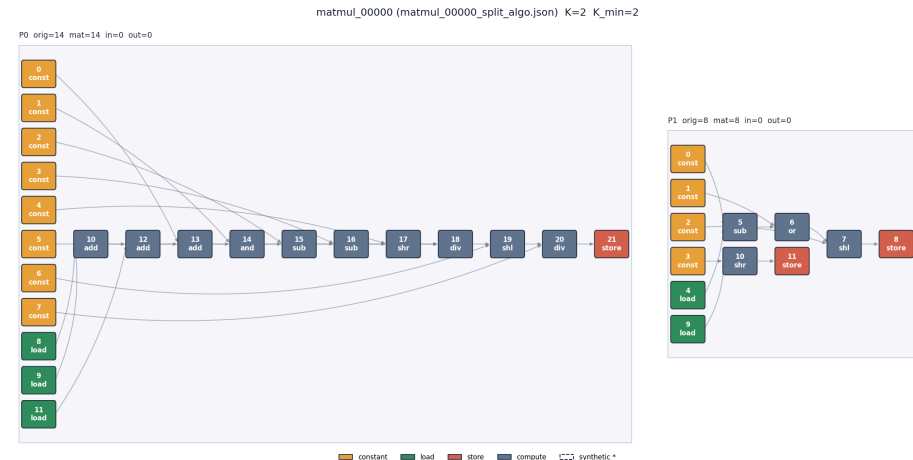


Figure 2: Partições one-shot

Cabe em **2 partições** num CGRA homogêneo 4×4 .

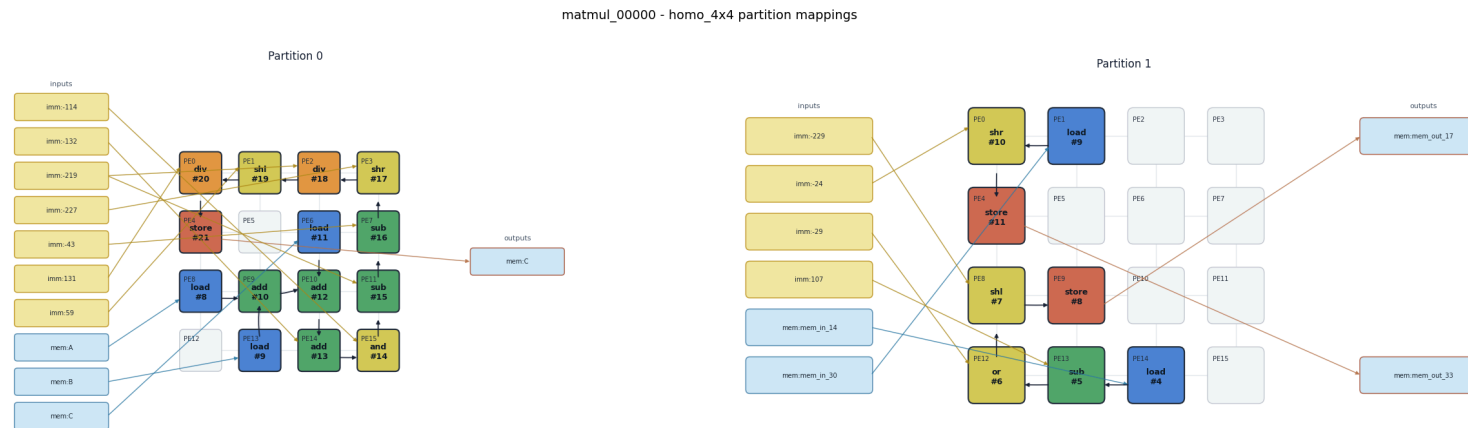


Figure 3: Mapping das partições no CGRA 4×4

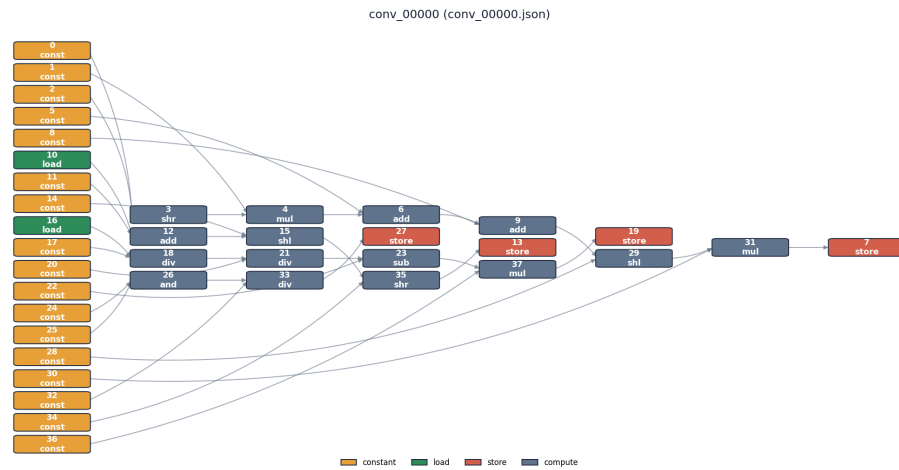


Figure 4: DFG sintético de convolução

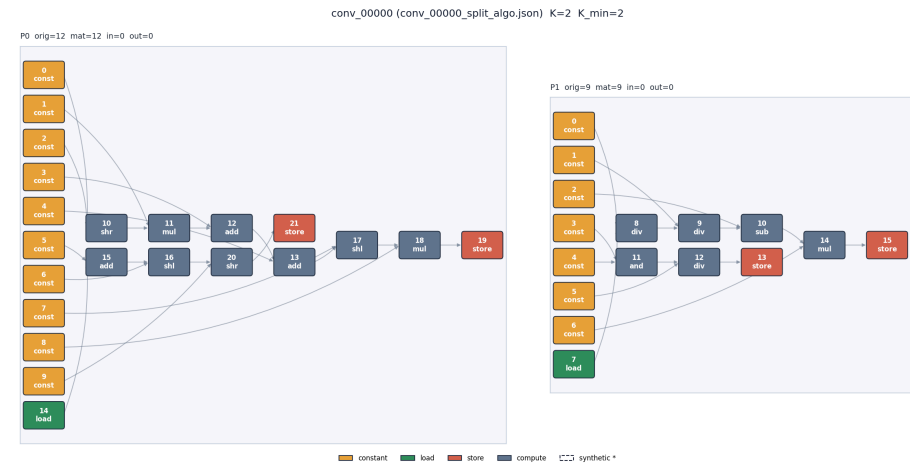


Figure 5: Partições one-shot

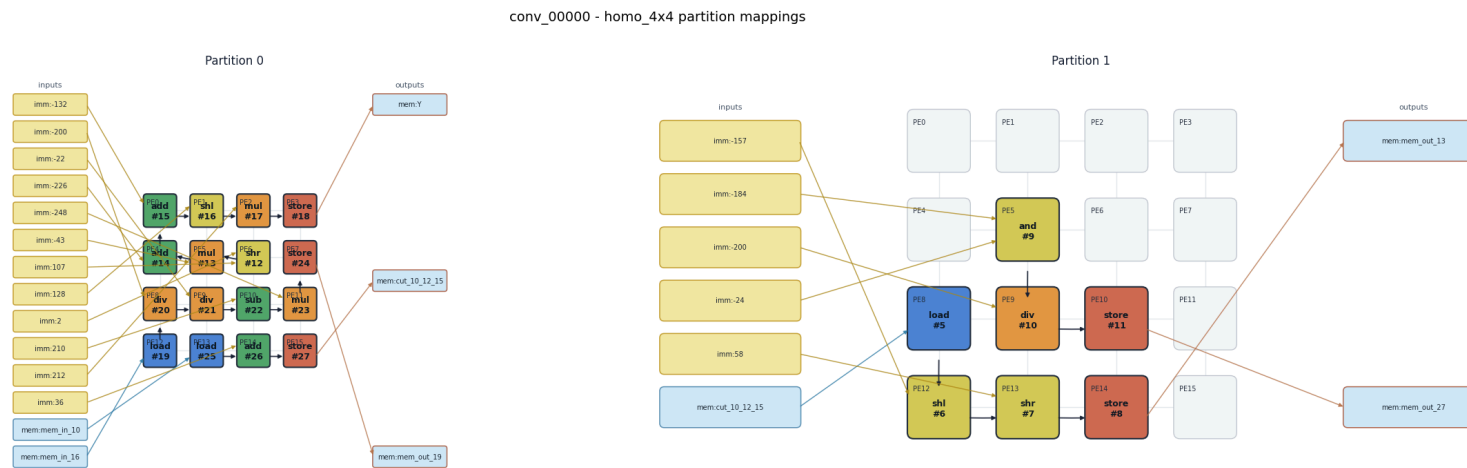


Figure 6: Mapping das partições no CGRA 4×4

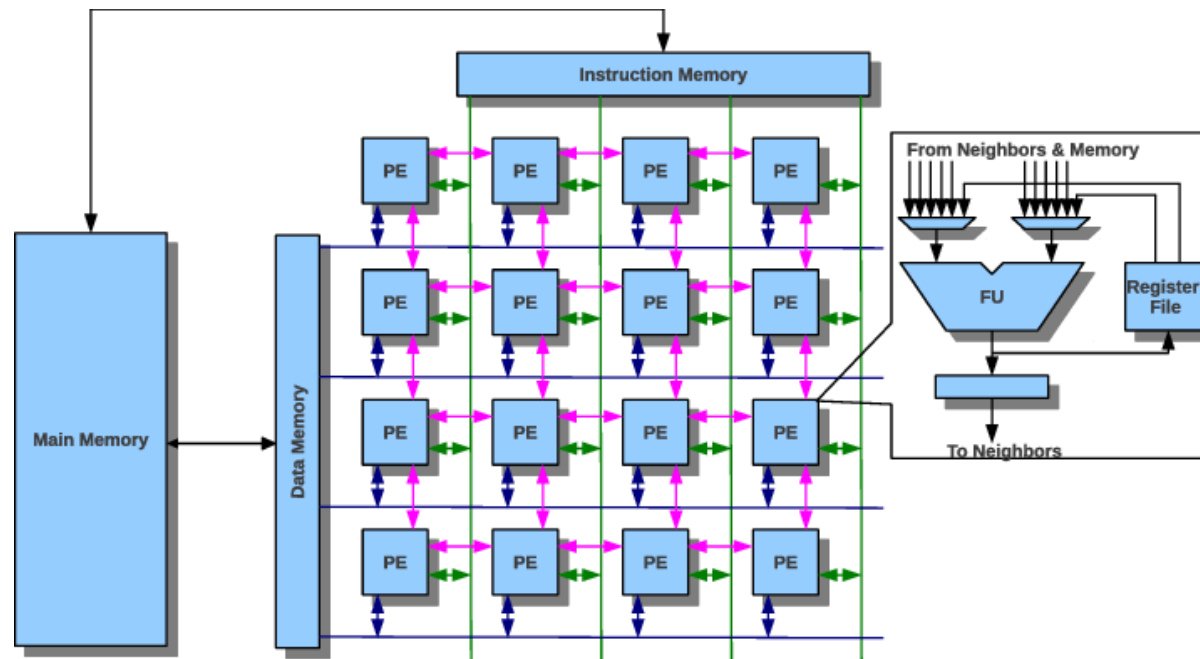


Figure 7: CGRA 4×4 : grid de PEs com memória de dados e instruções; cada PE é uma ALU com register file local. Figura de Jeyapaul *et al.* [1]

1. **Comparação** — pensar numa equivalência mais formal entre o cenário de one-shot e o de modulo scheduling, para comparar com a abordagem anterior
2. **Escrita do relatório final** — capítulos de validação e conclusões ainda por fechar

REFERENCES

- [1] R. Jeyapaul, S. Marathe, and A. Shrivastava, “Enabling Multithreading on CGRAs,” in *2011 International Conference on Parallel Processing*, IEEE, 2011, pp. 255–264.